

Synthesis in coreBuilder and coreConsultant

coreBuilder Application Note

Mark Noll, Synopsys Design Reuse Group

July 2000

Abstract

This application note defines a methodology for synthesizing designs in coreConsultant. It covers material for both the core developer and the core integrator. A process will be defined for taking both new and legacy cores through the process of constraint definition, specification of synthesis intent that drives the script generation process, and then actually synthesizing a core. This document also provides a step by step process to follow when the resulting QoR does not meet the specified constraints.

Introduction

There are three primary steps that must be followed in order to synthesize a design using coreConsultant. First, the design must be properly constrained. Constraints include information such as input arrival times, maximum area, and point-to-point timing exceptions. Second, information must be provided to control the synthesis script generation process within coreConsultant. Finally, a synthesis strategy is chosen, and synthesis scripts are generated. In most cases, all of the information except for the synthesis strategy are provided by the core developer, and then overridden when necessary by the core integrator. The outline of this document follows these three steps, and includes a final section on approaches to take when constraints are not being met, and an appendix of problems to watch out for.

In most cases, the core integrator should simply be able to specify contextual constraints, select a synthesis strategy, and get the results they need. This is because the core developer should have specified synthesis intent (constraints and strategy control attributes) for optimal synthesis results. Each sub-section is tagged as applying to either core development, core integration, or both. Core developers should read this entire document carefully and strive to follow the methodology as closely as possible. Core integrators should only need to look at the sections marked as applying to integration.

There are a number of tables of attributes in this document. In order to provide a focus on the most important attributes, the attributes that are most important to focus on are shown in bold in each of the tables.

This document is focused on the primary RTL-to-gates flow and therefore does not discuss any of the following topics: test insertion, clock gating, or timing model generation.

Constraint Specification

Careful and complete specification of constraints is critical for obtaining good synthesis results. Constraints should be fully specified for all designs (and their respective ports) that are going to be individually compiled. This set of designs defines the 'compile frontier', a topic discussed in detail in the 'Selecting the Compilation Frontier (development)' section later in this document. If constraints are not specified on the leaf level designs, then core integrators should be directed to utilize design budgeting for **all** initial compiles. One thing that simplifies the process of setting attributes is the concept of attribute inheritance. Most attributes support automatic inheritance. For example an input delay set on a top-level port will automatically be set on all directly connected lower level ports. The table below lists the attributes that should be set. For additional information about any of these attributes, please refer to the associated man page. In addition to the standard constraints, point to point timing exceptions (which should be avoided at all costs from the design perspective) should be specified if they do exist in the design.

Design Attributes	Input Port Attributes	Output Port Attributes
MaxArea	DrivingCell	MaxOutputDelay
	InputDelay	MaxOutputDelayFallingEdge
	MaxInputDelay	MinOutputDelay
	MaxInputDelayFallingEdge	MinOutputDelayFallingEdge
	MinInputDelay	OutputDelay
	MinInputDelayFallingEdge	PinLoadCount
		PinLoadType

Initial Definition for New Cores (development)

Constraints for new cores can be written in one of two ways. First, constraints can be written using the coreBuilder command syntax (set_port_attribute, set_design_attribute, etc.) and stored in TCL intent

files associated with each design. For example, constraints for design top would go in the file 'top.tcl'. When using this approach you can take advantage of utility commands such as `all_designs`, `all_inputs`, and `all_outputs`. Note also that the `all_inputs` and `all_outputs` commands support options for restricting the port list. In particular, you can use the `-no_primary` switch when specifying port intent on sub-design ports so that you only set values on sub-design ports that are not inheriting values from high level ports. The table in appendix A indicates the corresponding DC commands for those who are used to that syntax. Constraints should always be specified in a technology portable format using the formulas available for each constraint attribute. For example, input delay should be specified using the 'percent_of_period' formula and driving cells should be specified using 'select_by_function' or 'select_by_class'.

Constraints can also be written using version 1.1 of the Synopsys Design Constraints (SDC) syntax. This is a TCL-based syntax common to both Design Compiler and PrimeTime for specifying constraint information. See the Design Compiler `write_sdc` man page for details of the syntax. When using the SDC syntax, constraints for design top should be stored in a file named 'top.sdc'.

Initial Definition for Legacy Cores (development)

For legacy cores, the same approach as defined above for new cores should be used. Existing `write_script` output from Design Compiler can be used as a guide to values of existing attributes, but as was stated previously, values should be made technology independent.

Another approach to defining constraints for coreBuilder is to export them from Design Compiler in SDC format. This is not the preferred approach during core development since the values are technology dependent. See the next section of the document for when this is appropriate.

Specifying Contextual Constraints (integration)

When a core is going to be integrated into a design, it has a particular usage context, and is being synthesized into a particular technology. Since none of this information is available when the core is being developed, these constraints must be specified during core integration. Contextual constraints fall into two categories. The first includes technology specific information such as wireload models, operating conditions, DRC constraints, and definition of the clock waveform. The second includes information such as context specific values for most constraints on the top-level ports of the design. In many cases the core developer can specify reasonable defaults for these constraints, particularly when signals into and out of the core are registered. However, values for input and output delay and other constraints often need to be specified specifically for the context the core is being used in.

If the design has already been integrated once, an alternative approach to generate accurate context and technology related constraints for synthesis is to use the `write_sdc` command in Design Compiler to generate accurate top level constraints for coreConsultant, which can then be read in using the `read_sdc` command. Please see the coreConsultant man page for `read_sdc` regarding limitations in terms of supported commands and options.

The following table lists the technology specific constraints that are not typically set until core integration time.

Design Attributes	Port Attributes	Clock Attributes
MaxCap	MaxCap	ClockFallLatency
MaxFanout	MinCap	ClockHoldUncertainty
MinCap	MaxFanout	ClockRiseLatency
OperatingConditionsBest	MaxTransition	ClockSetupUncertainty
OperatingConditionsWorst		ClockSourceFallLatency
ParentWireLoad		ClockSourceRiseLatency
WireLoad		CycleTime

WireLoadGroup	FixHold
WireLoadMinBlockSize	InterClockFallUncertainty
WireLoadMode	InterClockRiseUncertainty
	MaxTransitionDelay
	MinTransitionDelay
	Waveform

Controlling the Synthesis Process

Each of the following sections defines a different aspect of control over the synthesis script generator. Within each section a number of related attributes will be defined which provide control over a particular function or functions within the synthesis script generator.

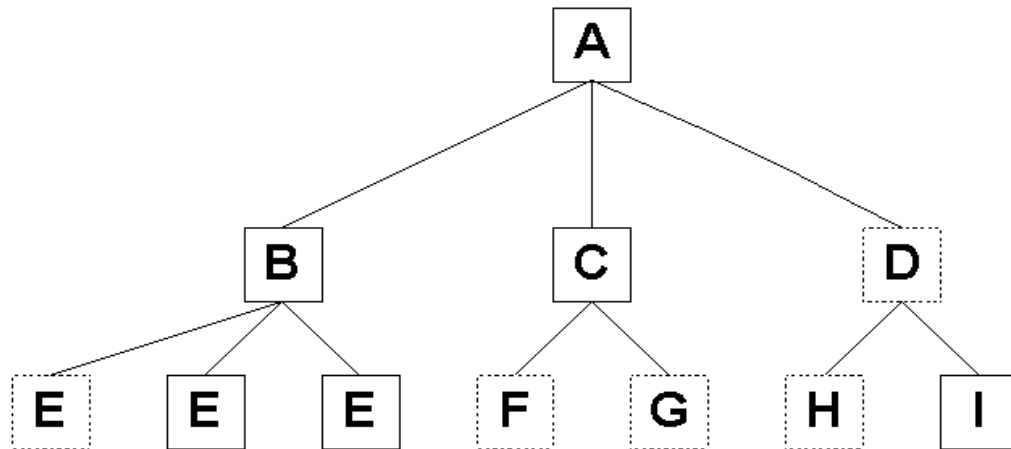
Selecting the Compilation Frontier (development)

The default compile strategy for coreConsultant is top-down. However, many designs are too large to support this strategy or else take too long to warrant the benefits of a top-down compile. In these cases a bottom-up or partial bottom-up strategy is preferred. Proper setting of a handful of attributes allows for any compile strategy between fully top-down and fully bottom-up. The following table defines the design attributes that define which designs will be compiled, and the order in which they are compiled, by Design Compiler (i.e. which designs make up the compilation frontier).

Attribute	Description
MapBlockIndividually	Indicates that the given design should be compiled individually (i.e. on the compile frontier).
MapSubblocksIndividually	Indicates that each of the immediate sub-designs of the given design should be compiled individually. Equivalent to setting MapBlockIndividually on each immediate sub-design.
CompileOrder	Indicates the order in which to compile designs within the given design. This attribute should be set to a list of instance names within the given design and indicates (for that that are going to

	be compiled) the order in which the compile should occur. By default, designs are compiled in order based on the percentage of ports that are connected directly to the top design, under the assumption that these designs are inherently more accurately constrained. This attribute should not typically need to be set.
Uniquify	If TRUE (the default) the design will be uniquified before any compiles occur. If FALSE, this design will be compiled individually, independent of the settings of MapBlockIndividually on this design, or MapSubblocksIndividually on the parent design.
RepresentativeContext	Applies only to designs that Uniquify FALSE. Indicates which cell instance should be characterized prior to compiling the non-unique design. The value should be a full instance name relative to the top design.

As a general guideline, set MapBlockIndividually on each design to be compiled individually. Also, set Uniquify to FALSE on designs with large numbers of instances if those designs are large enough to merit individual compilation. Consider the following design hierarchy:



The designs shown with dashed lines are to be compiled individually. The instance names of the three E instantiations are E1, E2, and E3 respectively, and all other instance names match the design name. Then the following intent should be specified:

```

set_design_attribute -design H MapBlockIndividually TRUE
set_design_attribute -design D MapBlockIndividually TRUE
set_design_attribute -design C MapSubblocksIndividually TRUE
set_design_attribute -design E Uniquify FALSE
set_design_attribute -design E RepresentativeContext B/E1

```

The following designs will then be compiled: E, F, G, H, D, A. The top design implicitly has MapBlockIndividually set on it. The order of compilation within sibling designs (like F and G) is determined by the tool unless the CompileOrder attribute is set on design C to force a specific order. Otherwise the only guarantee is that lower level blocks are compiled before their parent blocks. The CompileOrder attribute should not typically need to be set since the tool figures out an intelligent ordering based on how accurately each design is constrained.

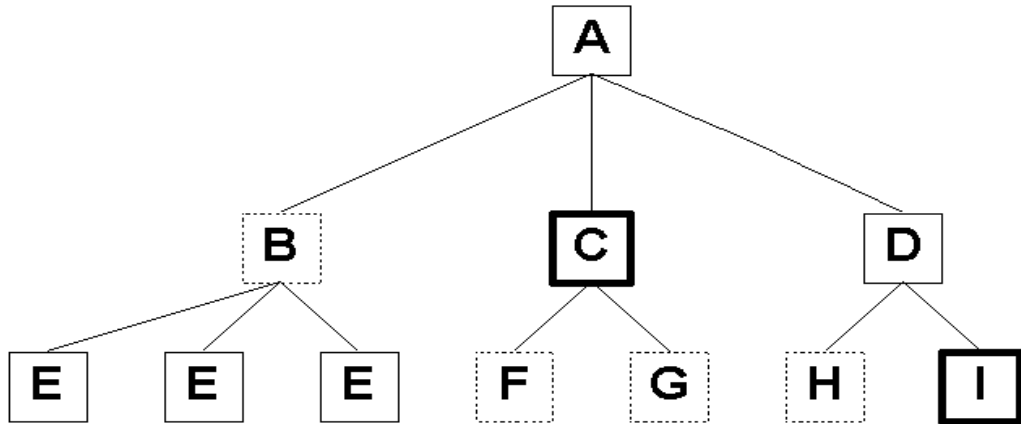
Ungrouping and Hierarchy Preservation (development)

This section of the document defines attributes used to control what the final design hierarchy is going to look like. These attributes play a large role in the final QoR of the design so it is important to understand exactly how they work. The table below defines the design attributes associated with ungrouping.

Attribute	Description
PreserveHierarchy	Indicates that the given design should never be ungrouped.
PreserveBoundaries	Indicates that the boundaries of the given design should be preserved. Implies PreserveHierarchy and prevents boundary optimization by Design Compiler.
UngroupSubblocks	Indicates that the given design should have all of its sub-designs ungrouped prior to the initial compile. Any sub-designs with PreserveHierarchy set to TRUE will be preserved so it is possible to do partial ungrouping.
Ungroup	Indicates that the given design should be ungrouped whenever a containing design is compiled.

Each of these attributes defaults to FALSE, so by default, blocks are not automatically ungrouped prior to compile, but

they might get ungrouped prior to one of the incremental compiles if they are small enough. As a general guideline, set UngroupSubblocks wherever possible, as it will improve QoR. The PreserveHierarchy and PreserveBoundaries attributes should be set only when absolutely necessary to preserve a hierarchy or particular boundary. As an example, consider the following hierarchy:

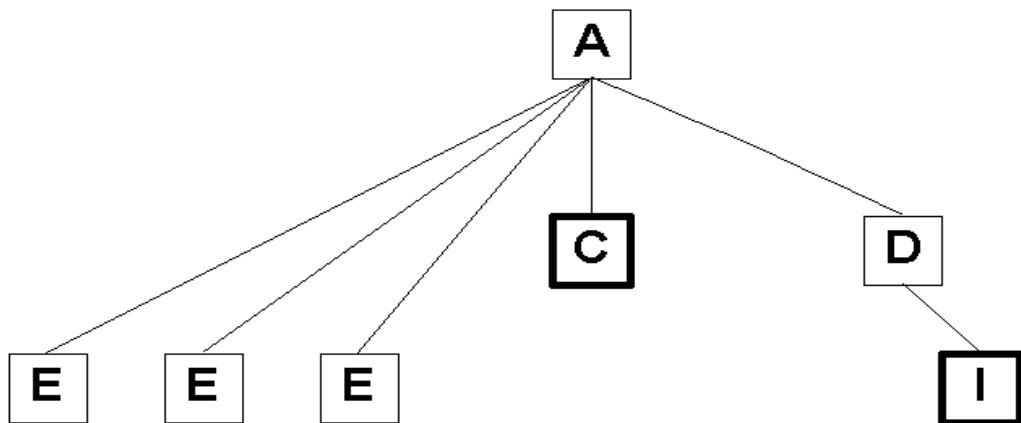


Assume that the blocks shown with dashed lines should be ungrouped and that we want to preserve the hierarchy of the blocks shown in bold. This can be accomplished with the following intent.

```

set_design_attribute -design B Ungroup TRUE
set_design_attribute -design C UngroupSubblocks TRUE
set_design_attribute -design C PreserveHierarchy TRUE
set_design_attribute -design D UngroupSubblocks TRUE
set_design_attribute -design I PreserveHierarchy TRUE
  
```

Just after the initial compile, the hierarchy will look like the following:



During the incremental compiles, the only block that can now be ungrouped is D (assuming E has not been uniquified). This will occur if D is under the ungrouping threshold.

Design Characteristics (development)

The attributes in the design characteristics category describe the content of the different designs and ports. These attributes should all be set whenever they apply to any design or port on or above the compile frontier.

Attribute	Description
CanFlatten	Indicates that the given design might benefit from flattening to sum-of-products form. This is flattening of levels of logic within a single hierarchy, and does not change the logical hierarchy in any way.
HasArithmetic	Indicates that the given design contains arithmetic components and might benefit from special arithmetic oriented optimization such as BOA (assuming appropriate DC licenses area available)
HighFanout	Indicates that the given port is likely to have high fanout and might benefit from special buffer optimizations such as buffer balancing.
IsPipelined	Indicates that the given design is pipelined and might benefit from special optimization such as BRT (assuming appropriate DC licenses area available).
PreserveMapping	Indicates that the given design is already mapped and should not be modified in any way. This is essentially equivalent to putting a dont_touch on the design.
PreserveStructure	Indicates that the given design contains structuring that should be preserved and therefore no structural optimization should be done.
PurelyStructural	Indicates that the given design contains only structural RTL code and does not need to be compiled during the initial (technology mapping) compile step. Setting this where applicable will help with run time, but has no impact on QoR.

Tuning for Context (development and integration)

Just as in the case of constraints, there is a set of attributes controlling the script generator that are somewhat technology and /or context dependent. With one exception, these attributes should probably not be specified unless constraints are not being properly satisfied. The exception is a critical one as it probably has more impact on the generated strategy than any other attribute. The attribute `OptimizationPriorities` should be set on each design on the compile frontier. Note that it does not have to be set to the same value on every design. The following table lists the context/technology specific strategy control attributes.

Attribute	Description
<code>CriticalPathCrank</code>	Indicates that when doing a high effort compile and timing is not being met, an extra attempt at critical path resynthesis (CPR) should be tried, specifically on paths within the given design that have the worst timing slack. This is an expensive operation.
<code>CriticalTiming</code>	Indicates that the given port is timing critical and causes it to be inserted into its own path group during timing focused optimization.
OptimizationPriorities	Specifies goals and the relative importance of the goals for the given design. Can be set to 'timing', 'timing_area', 'area_timing', or 'area'. It is critical that this is set properly on each design being compiled.
<code>Overconstrain</code>	Indicates that during compilation the given design should be over-constrained by the specified percentage of the cycle time (value should be between 0 and 1). This is useful in certain situations for severely timing critical designs.
<code>Underconstrain</code>	Indicates that during compilation the given design should be under-constrained by the specified percentage of the cycle time (value should be between 0 and 1). This is useful in certain situations for area critical designs that easily met timing.

Using the Synthesis Strategies

During Core Development

After specifying all synthesis intent as completely as possible as defined in the previous sections of the document, the best approach is to try a simple default compile. This will do an initial compile of everything on the compile frontier, followed by one incremental compile. If that does not give reasonable results, then try again with the QoR effort set to 'high'. This will enable more aggressive optimization during the first two compile passes, and will add a second incremental compile. See the next section 'What to do When QoR Goals are not Met' if the design is still not meeting constraints.

During Core Integration

The first step is to ensure that all of the context-based intent has been specified as accurately as possible, as defined in the previous sections of the document. If the core developer provided suggestions about settings for the synthesis strategy (QoR Effort, etc.), then these should be followed as closely as possible. Otherwise, follow the guidelines given above for the core developer.

What to do When QoR Goals are not Met

The following sections provide ideas for changes that can be made to help improve different aspects of the QoR of a design.

Improving Timing and Area

- Make sure all strategy control attributes have been specified as accurately as possible.

- If a newer version of Design Compiler is available, switching DC versions is a simple way to improve QoR. Be sure to regenerate the synthesis scripts targeting the new version prior to running the scripts.
- If you have access to DC-Ultra but have not been using it yet (keys not visible or explicitly disabled in Synthesize activity), enabling this feature and regenerating the synthesis scripts will provide improved QoR.
- Review settings of the ungrouping and hierarchy preservation attributes (UngroupSubblocks, Ungroup, PreserveHierarchy, PreserveBoundaries). Try to set the first two TRUE as much as possible and the latter two FALSE (the default) as much as possible. Removing logical boundaries reduces restrictions on what Design Compiler can optimize.
- If you have Uniquify set to FALSE on a number of designs, consider setting it to true. If Uniquify is set to FALSE on small designs, consider leaving Uniquify FALSE, but also setting Ungroup on the design.

Improving Just Timing

- Change OptimizationPriorities to be more timing focused. For example, changing from 'timing_area' to 'timing', or from 'area' to 'timing_area' on designs that are not meeting constraints.
- If there are just a few endpoint ports causing problems, try setting the CriticalTiming attribute on these ports.
- Try setting the Overconstrain attribute on those designs that are causing problems. This should not typically be set to over constrain by more than 15% (i.e. value ≤ 0.15).
- Try setting the CriticalPathCrank attribute on those designs that are causing problems. Note that this can cause a large increase in run time.

Improving Just Area

- Change OptimizationPriorities to be more area focused. For example, changing from 'timing_area' to 'area', or from 'timing' to 'timing_area' on designs that are not meeting constraints.

Improving Run Time

- Set the MaxArea attribute on designs on or above the compile frontier. Unless this attribute is set, the script generator uses 'set_max_area 0' for all designs except those with OptimizationPriorities set to 'timing', which do not get an area constraint.
- Make sure PurelyStructural is set to TRUE on designs that contain no logic (i.e. only instantiations of other designs).
- Lower the compile frontier. Note that this may have a negative impact on QoR.
- Set Uniquify to FALSE on multiply instantiated designs.
- Disable the DC-Ultra license in the Synthesize 'Licenses' tab. Note that this will have a negative impact on QoR.

Appendix – Problems to Watch Out For

- Results get worse between 'initial' and 'incr1' or between 'incr1' and 'incr2'.

This problem is caused by setting WireLoadMinBlockSize to a value that is too small. There is an ungrouping step that occurs just prior to the two incremental compiles. If ungrouping a design causes its wireload model to change dramatically, then the timing of the design can change significantly simply by doing the ungrouping. Ideally, the WireLoadMinBlockSize attribute should be set to a value just above the size of the largest design that is going to be

ungrouped. This will prevent any change in timing during ungrouping.

- Design contains unmapped logic after the initial compile.

This is typically caused by setting the `PurelyStructural` attribute on a design that contains logic. This attribute should only be set on designs that contain hierarchical cell instantiations and wires.